

Professional Coding Specialist

COS Pro 파이썬 2 급

14 강- 17 강. 모의고사 3 차

모의고사 3 차 문제 1-10 번

과정 소개

모의고사(YBM 샘플) 문제를 통해 실전에 대비할 수 있다. (회차당 10 문항, 각 문항별 문제 풀이)

학습 목차

모의고사 3 차 문제 1 번
모의고사 3 차 문제 2 번
모의고사 3 차 문제 3 번
모의고사 3 차 문제 4 번
모의고사 3 차 문제 5 번
모의고사 3 차 문제 6 번
모의고사 3 차 문제 7 번
모의고사 3 차 문제 8 번
모의고사 3 차 문제 9 번
모의고사 3 차 문제 10 번

학습 목표

1. YBM IT(www.ybmit.com) 에서 제공하는 COS Pro 파이썬 2 급 샘플 문제 메뉴의 모의고사를 풀어보며 COS Pro 파이썬 2 급 시험에 대비할 수 있다.

1

모의고사 3 차 문제 1 번

문제 1 의 정답

n 번 학생의 등수 구하기

```
def func_a(scores, score):  
    rank = 1  
    for s in scores:  
        if s == score:  
            return rank  
        rank += 1  
    return 0
```

```
def func_b(arr):  
    arr.sort(reverse=True)
```

```
def func_c(arr, n):  
    return arr[n]
```

```
def solution(scores, n):  
    ① s = func_c(scores, n)  
    ② func_b(scores)  
    ③ answer = func_a(scores, s)  
    return answer
```

```
scores = [20, 60, 98, 59]  
n = 3  
ret = solution(scores, n)
```

- ① n 번 학생의 점수를 변수에 저장합니다.
- ② 점수를 내림차순으로 정렬합니다.
- ③ 리스트의 첫 번째 원소부터 마지막 원소까지 순회하며 n 번 학생의 점수를 찾습니다.
3-1. 1 번 단계에서 저장해 둔 점수와 같은 점수를 찾으면 등수를 return 합니다

- ① n 번 학생의 점수를 미리 찾아서 s 라는 변수에 저장해 둔다.

```
def func_c(arr, n):  
    return arr[n]
```

- 메인에서 scores 리스트에 있는 점수를 arr 리스트로, n 의 숫자값을 n 매개변수로 보내고, arr[n]을 리턴함으로써 n 번 학생의 점수를 s 라는 변수에 저장한다.

```
s = func_c(scores, n)
```

- ② 리스트의 큰 값부터 정렬한다.

- 리스트의 sort(reverse=True)메소드를 이용하여 점수를 내림차순으로 정렬한다. 함수가 아닌 메소드의 경우에는 내림차순으로 정렬한 후 데이터를 완전히 변경한다.

```
def func_b(arr):  
    arr.sort(reverse=True)
```

- 따라서 메인에서 새로운 변수에 정렬한 결과를 넣어줄 필요가 없음을 주의한다.

```
func_b(scores)
```

- ③ n 번 학생의 등수를 구한다.
- scores 리스트는 이미 값이 큰 순서대로 정렬된 상태이므로 큰 값부터 순서대로 정렬이 되어 있다. 따라서 n 번 학생의 점수가 몇 번째 있는지 찾으면 그 값이 등수가 되므로 아래와 같이 코드한다.
- rank를 1 위로 초기화 한다.
- scores 리스트에 있는 점수를 하나씩 s 에 가져온다.
- 현재 n 번의 점수를 score 매개변수에 받아서 n 번의 점수와 같은 값을 만나면 해당 rank 를 리턴한다.
- n 번의 값과 같은 값이 없으면 rank 를 계속 1 씩 증가시킨다.

```
def func_a(scores, score):  
    rank = 1  
    for s in scores:  
        if s == score:  
            return rank  
    rank += 1  
    return 0
```

```
answer = func_a(scores, s)
```

<정렬 알고리즘 참고>

인강에서 설명하고 있는 다양한 정렬 알고리즘 참고.

- [2 급 3 차 1 번 알고리즘 - 순위구하기 1.py]
- [2 급 3 차 1 번 알고리즘 - 순위구하기 2.py]
- [2 급 3 차 1 번 알고리즘 - 순위구하기 3-1.py]
- [2 급 3 차 1 번 알고리즘 - 순위구하기 3-2.py]

COS PRO PYTHON 2급

알고리즘

n번 데이터의 순위 구하기 1

[2급 3차 1번 알고리즘 - 순위구하기1.py]

- 1 새로운 ranking 리스트의 모든 데이터를 1등으로 정한다.

0	1	2	3
1	1	1	1

- 2 기준으로 정한 데이터와 데이터들을 모두 비교한다.
기준보다 비교한 값이 더 크면, 순위를 +1해서 새로운 리스트(ranking)에 저장한다.

0	1	2	3
20	60	98	59

scores 리스트 : 점수

ranking 리스트 : 순위

- 3 n번 데이터의 순위를 구한다. → ranking[n]

ranking 리스트 : 순위

```
def solution(scores, n):
    1 ranking=[1]*len(scores)
    2 for i in range(len(scores)):
        for j in range(len(scores)):
            if scores[i] < scores[j]:
                ranking[i]+=1
    print(ranking)
    3 answer=ranking[n]
    return answer
```

0	1	2	3
1	1	1	1

0	1	2	3
4	2	1	3

0	1	2	3
20	60	98	59
0	1	2	3
4	2	1	3

COS PRO PYTHON 2급

알고리즘

n번 데이터의 순위 구하기 2

[2급 3차 1번 알고리즘 - 순위구하기2.py]

- 1 인덱스 번호를 처음부터 끝까지 돌면서 기준으로 정한 데이터의 rank(순위)를 무조건 1등으로 정한다.

rank : 순위

3

- 2 기준으로 정한 데이터와 데이터들을 모두 비교해서 기준보다 큰 데이터가 있으면 순위를 +1해서 새로운 리스트(ranking)에 추가한다.

0	1	2	3
20	60	98	59

scores 리스트 : 점수

ranking 리스트 : 순위

- 3 n번 데이터의 순위를 구한다. → ranking[n]

```
def solution(scores, n):
    ranking=[]
    1 for i in range(len(scores)):
        rank=1
        2 for s in scores:
            if scores[i] < s:
                rank+=1
            ranking.append(rank)
    3 print(ranking)
    answer=ranking[n]
    return answer
```

COS PRO PYTHON 2급

알고리즘

n번 데이터의 순위 구하기 3

[2급 3차 1번 알고리즘 - 순위구하기3-1.py]
[2급 3차 1번 알고리즘 - 순위구하기3-2.py]

scores 리스트 : 점수

0	1	2	3
20	60	98	59

- ① n번 데이터 값을 저장한다. (n=3)

s_n

59

- ② 전체 데이터를 정렬한다.

0	1	2	3
98	60	59	20

- 3-1 n번 값(s_n)과 같은 값을 찾아
인덱스 번호+1 로 순위를 구한다.

- 3-2 rank를 1에서 시작해서
n번 데이터와 같은 값을 찾을 때까지 rank
를 1 증가시킨다.

```
def solution(scores, n):
```

```
    ① s_n = scores[n]
```

```
    ② scores.sort(reverse=True)
```

```
    3-1 rank=0
```

```
    for i in range(len(scores)):
```

```
        if s_n==scores[i]:
```

```
            rank=i+1
```

```
            return rank
```

```
    return 0
```

3-2

```
    rank=1
```

```
    for c in scores:
```

```
        if s_n ==c:
```

```
            return rank
```

```
            rank+=1
```

```
    return 0
```

2

모의고사 3 차 문제 2 번

문제 2 의 정답

장학생 수 구하기

```

def func_a(current_grade, last_grade, rank, max_diff_grade):
    arr_length = len(current_grade)
    count = 0
    for i in range(arr_length):
        if current_grade[i] >= 80 and rank[i] <= arr_length // 10:
            count += 1
        elif current_grade[i] >= 80 and rank[i] == 1:
            count += 1
        elif max_diff_grade > 0 and max_diff_grade == current_grade[i] - last_grade[i]:
            count += 1
    return count

def func_b(current_grade):
    arr_length = len(current_grade)
    rank = [1] * arr_length
    for i in range(arr_length):
        for j in range(arr_length):
            if current_grade[i] < current_grade[j]:
                rank[i] += 1
    return rank

def func_c(current_grade, last_grade):
    max_diff_grade = 1
    for i in range(len(current_grade)):
        max_diff_grade = max(max_diff_grade, current_grade[i] - last_grade[i])
    return max_diff_grade

def solution(current_grade, last_grade):
    ❶ rank = func_b(current_grade)
    ❷ max_diff_grade = func_c(current_grade, last_grade)
    ❸ answer = func_a(current_grade, last_grade, rank, max_diff_grade )
    return answer

```



```
current_grade = [70, 100, 70, 80, 50, 95]
last_grade = [35, 65, 80, 50, 20, 60]
ret = solution(current_grade, last_grade)
```

- ① 이번 학기 성적을 기준으로 학생별 석차를 구합니다.
- ② 각 학생의 (이번 학기 성적 - 직전 학기 성적) 중 최댓값을 구합니다.
- ③ 아래 조건을 만족하는 학생을 발견하면 장학생 수를 1 증가시킵니다.
 - 3-1. 이번 학기 성적이 80 점 이상이고, 석차가 상위 10% 이내인 경우
 - 3-2. 또는 이번 학기 성적이 80 점 이상이고, 석차가 1 등인 경우
 - 3-3. 또는 (이번 학기 성적 - 직전 학기 성적)이 2 단계에서 구한 값과 같고, 그 값이 양수인 경우
- ④ 4. 장학생 수를 return 합니다.

- ① 이번 학기 성적을 기준으로 학생별 석차를 구합니다.

<func_b 함수>

```
def func_b(current_grade):
    arr_length = len(current_grade)
    rank = [1] * arr_length
    for i in range(arr_length):
        for j in range(arr_length):
            if current_grade[i] < current_grade[j]:
                rank[i] += 1
    return rank
```

rank 리스트의 순위를 모두 1로 초기화한다.

i 인덱스 번호를 기준으로 나머지 모든 데이터를 j 첨자를 이용해 비교해서 더 큰 값이 있으면 순위를 1 증가시켜 i 기준에 해당하는 값의 순위를 정한다.

이 문장을 종료하면 아래와 같은 리스트가 생성된다.

rank 리스트 : 순위

0	1	2	3	4	5
4	1	4	3	6	2

② 각 학생의 (이번 학기 성적 - 직전 학기 성적) 중 최댓값을 구합니다.

<func_c 함수>

```
def func_c(current_grade, last_grade):
    max_diff_grade = 1
    for i in range(len(current_grade)):
        max_diff_grade = max(max_diff_grade, current_grade[i] - last_grade[i])
    return max_diff_grade
```

최댓값을 1로 초기화 한다.

모든 데이터를 돌면서 각 학생의 (이번 학기 성적 - 직전 학기 성적) 중 최댓값을 max() 함수를 이용하여 구한다.

③ 아래 조건을 만족하는 학생을 발견하면 장학생 수를 1 증가시킵니다.

3-1. 이번 학기 성적이 80 점 이상이고, 석차가 상위 10% 이내인 경우

3-2. 또는 이번 학기 성적이 80 점 이상이고, 석차가 1 등인 경우

3-3. 또는 (이번 학기 성적 - 직전 학기 성적)이 2 단계에서 구한 값과 같고, 그 값이 양수인 경우

<func_a 함수>

```
def func_a(current_grade, last_grade, rank, max_diff_grade):
    arr_length = len(current_grade)
    count = 0
    for i in range(arr_length):
        if current_grade[i] >= 80 and rank[i] <= arr_length // 10:
            count += 1
        elif current_grade[i] >= 80 and rank[i] == 1:
            count += 1
        elif max_diff_grade > 0 and max_diff_grade == current_grade[i] - last_grade[i]:
            count += 1
    return count
```

3

모의고사 3 차 문제 3 번

문제 3 의 정답

체조 점수 구하기

“가장 높은 점수와 가장 낮은 점수를 제외한 점수들의 평균 구하기”

```
def solution(scores):  
    answer = 0  
    answer=(sum(scores)-max(scores)-min(scores))/(len(scores)-2)  
    return answer
```

```
scores1 = [35, 28, 98, 34, 20, 50, 85, 74, 71, 7]  
ret1 = solution(scores1)
```

```
scores2 = [1, 1, 1, 1, 1]  
ret2 = solution(scores2)
```

- (모든 데이터의 총합 - 최댓값 - 최솟값) // (총 개수 - 2) 를 구하되 sum(), max(), min(), len() 함수를 사용한다.
- 여기서는 평균을 정수로 표현해야 하기 때문에 /(나누기) 가 아니라 //(몫) 연산자를 이용해야 함을 주의한다.
- 나누기(/)를 이용할 때는 아래와 같이 int() 함수를 사용해야 함을 유의한다.

```
answer=(sum(scores)-max(scores)-min(scores))/(len(scores)-2)  
answer=int(answer)
```

※ 참고 : 제곱 표현식

** 연산자

pow() 함수

예시) 2 의 세제곱 : 2**3 / pow(2,3)

4

모의고사 3 차 문제 4 번

문제 4 의 정답

오타 수정을 위해 바뀌어야 하는 문자 개수 구하기

```
def solution(words, word):
    count = 0
    for comp in words:
        for x, y in zip(comp, word):
            if x != y:
                count = count + 1
    return count
```

1. "CODE"는 바르게 적혔습니다.
 2. "COED"는 E 와 D 를 각각 D 와 E 로 바꾸면 됩니다.
 3. "CDEO"는 D, E, O 를 각각 O, D, E 로 바꾸면 됩니다.
- 따라서 바뀌어야 하는 문자는 총 5 개입니다.

words	word	return
["CODE", "COED", "CDEO"]	"CODE"	5

- words 리스트에서 단어를 하나씩 가져와서 comp 변수에 넣는다.
- comp 와 word 변수 안의 문자를 순서대로 하나씩 꺼내서 서로 다른 개수(count 변수)를 증가시킨다.
- 이때, comp 와 word 를 인덱스 번호가 같은 순서대로 가져오기 위해서 zip() 함수를 사용한다.

```
def solution(words, word):
    count = 0
    for comp in words:
        for x, y in zip(comp, word):
            if x != y:
                count = count + 1
    return count
```

[2 급 3 차 4 번 다른 코딩 제안.py]

```
for comp in words:
    for i in range(len(word)):
        if comp[i]!=word[i]:
            count+=1
```

zip() 함수를 사용하지 않고, 인덱스 번호를 이용해서 같은 인덱스 번호에 있는 데이터끼리 비교하는 코드이다.

5

모의고사 3 차 문제 5 번

문제 5 의 정답

총 교통비 계산하기

```
def solution(member_age, transportation):  
    1 if transportation == 'Bus':  
        adult_expense = 40000  
        child_expense = 15000  
    elif transportation == 'Ship':  
        adult_expense = 30000  
        child_expense = 13000  
    elif transportation == 'Airplane':  
        adult_expense = 70000  
        child_expense = 45000  
  
    2 if len(member_age) >= 10:  
        adult_expense = adult_expense * 0.9  
        child_expense = child_expense * 0.8  
  
    3 total_expenses = 0  
    for age in member_age:  
        if age >= 20:  
            total_expenses += adult_expense  
        else:  
            total_expenses += child_expense  
  
    return total_expenses
```

- ① 교통 수단별 요금표를 참고하여 교통 수단과 어른, 어린이에 맞는 기본 요금을 정한다.
- ② 여행객이 10 명 이상일 경우 어른은 10%, 어린이는 20%의 할인 요금을 적용하여 할인 후 내야 할 금액을 계산한다.
- 이 문제의 정답지는 위와 같이 제시되어 있으나, 결과값이 정수로 리턴되어야 하므로 int()함수를 아래와 같이 써 주는 것이 더 정확하다.

```
int(adult_expense*0.9)
```

```
int(child_expense*0.8)
```

- ③ 여행객의 나이를 하나씩 읽어 들어서 성인, 어린이에 따라 다른 요금을 총 교통비에 더한다.

6

모의고사 3 차 문제 6 번

문제 6 의 정답

타일 색칠하기

```
def solution(length):  
    answer = ''  
    com = 'RRRGGB'  
    if length%6 ==1 or length%6 ==2 or length%6 ==4:  
        answer = '-1'  
    else:  
        for i in range(length):  
            answer+=com[i % 6]  
    return answer
```

```
tile_length1 = 11  
ret1 = solution(tile_length1)
```

```
tile_length2 = 16  
ret2 = solution(tile_length2)
```

타일을 'R', 'G', 'B' 색으로 칠하려 합니다. R 색으로는 3 칸을 한 번에, G 색으로는 2 칸을 한 번에 칠할 수 있으며, B 색으로는 1 칸을 칠할 수 있습니다. 색은 R, G, B 순서로 한 번씩 번갈아 가면서 사용해야 하며, 타일의 길이를 넘겨서 칠할 수는 없습니다.

예를 들어, 타일 길이가 11 이면 "RRRGGBRRRG"의 색으로 칠할 수 있습니다.

COS PRO PYTHON 2급

[3차] 6번

타일 색칠하기 ★★★★★

빈 칸 채우기

타일 길이: 16 타일 패턴: RRRGGB (총 6개)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
R	R	R	G	G	B	R	R	R	G	G	B	R	R	R	-

칠할 수 있는 경우

1. 타일 전체 패턴이 6개이므로(RRRGGB), 타일 길이가 6의 배수이면 타일을 칠할 수 있음
(6으로 나눈 나머지가 0)
예) 타일 길이 6, 12, 18 ...
2. 타일 전체 패턴인 6개씩 칠하다가 마지막에 B 하나만 덜 칠하는 경우 (RRRG), 즉 타일 길이가 6의 배수에서 1개 모자랄 경우 칠할 수 있음
(6으로 나눈 나머지가 5)
예) 타일 길이 5, 11, 17 ...
3. 타일 전체 패턴인 6개씩 칠하다가 마지막에 G 두 개, B 하나 덜 칠하는 경우 (RRR), 즉 타일 길이가 6의 배수에서 3개 모자랄 경우 칠할 수 있음
(6으로 나눈 나머지가 3)
예) 타일 길이 3, 9, 15 ...

칠할 수 없는 경우

(6으로 나눈 나머지가 0, 3, 5 가 아님)
(6으로 나눈 나머지가 1, 2, 4)

```
def solution(length):
    answer = ''
    1 com = 'RRRGGB'
    2 if length%6 ==1 or length%6 ==2 or length%6 ==4:
        answer = '-1'
    else:
    3     for i in range(length):
        answer+=com[i % 6]
    return answer
```

i →	0	1	2	3	4	5	6	7	8	9	10
→	R	R	R	G	G	B	R	R	R	G	G

com의 인덱스 번호를 6으로 나눈 나머지

- ① 칠해야 하는 RGB 색의 패턴을 com 변수에 초기화 한다.
- ② 색을 칠할 수 없는 경우이면, 즉 타일 길이를 6 으로 나눈 나머지가 1, 2, 4 중 하나이면 -1 을 리턴한다.
- ③ 색을 칠할 수 있는 경우이면, 즉 타일 길이를 6으로 나눈 나머지가 0, 3, 5 이면, 타일의 길이 만큼 돌면서 한 글자씩 answer 에 붙인다.

7

모의고사 3 차 문제 7 번

문제 7 의 정답

주스를 만들 수 있는 최대 개수 구하기

```
def solution(num_apple, num_carrot, k):
    answer = 0

    1 if num_apple < num_carrot * 3:
        answer = num_apple // 3
    else:
        answer = num_carrot

    2 num_apple -= answer * 3
    num_carrot -= answer

    3 i = 0
    while k - (num_apple + num_carrot + i) > 0:
        if i % 4 == 0:
            answer -= 1
        i = i + 1

    return answer
```

〈알고리즘〉

1. 최대 나올 수 있는 잔(세트) 수 계산
2. 남은 개수 계산
3. 잔(세트)으로 묶은 후 k 개가 남지 않으면 잔(세트)을 하나씩 취소



- ① 사과 3 개와 당근 1 개로 주스 한 잔을 만들 수 있다. 최대로 만들 수 있는 주스 잔을 계산하여 answer 변수에 넣는다.

사과 개수가 당근 개수의 3 배보다 많으면 당근 개수가 최대로 만들 수 있는 주스 잔의 개수가 되고, 아니면 사과 개수를 3 으로 나눈 몫이 최대로 만들 수 있는 주스 잔의 개수가 된다.

사과 개수 >= 당근 개수 x 3 → 당근 개수
아니면 → 사과 개수//3

정답 코드에서는 이것을 거꾸로 적은 것이며, 이해를 쉽게 하기 위해 설명을 거꾸로 작성하였음.

- ② 최대로 만들 수 있는 잔을 만들고 나서 남은 개수를 계산한다.

남는 사과 개수 = 총 개수 - 세트(잔) 개수 x 3

남는 당근 개수 = 총 개수 - 세트(잔) 개수

- ③ 주스를 만들 세트(잔)로 묶은 후 총 개수가 k(토끼에게 줄 먹이 개수)보다 작으면 한 세트씩 계속 취소한다.

[2 급 3 차 7 번 다른 코딩 제안.py]

```
def solution(num_apple, num_carrot, k):
    answer = 0
    if num_apple >= num_carrot * 3:
        answer=num_carrot
    else:
        answer=num_apple//3

    num_apple2 = num_apple - answer*3
    num_carrot2 =num_carrot - answer

    leftover=num_apple2+num_carrot2

    while k > leftover:
        answer-=1
        leftover+=4

    return answer
```

세트로 묶고 나서 남은 먹이가 토끼에게 줄 먹이 k 보다 작으면 세트를 하나씩 취소하면서 남은 음식의 개수에 4 개씩 증가하는 방법으로 코드한다.

8

모의고사 3 차 문제 8 번

문제 8 의 정답

TV 두 대 이상을 트는 총 시간 구하기

```
def solution(programs):
    answer = 0
    1 used_tv = [0] * 25

    2 for program in programs:
        for i in range(program[0], program[1]):
            used_tv[i] = used_tv[i] + 1

    3 for i in used_tv:
        if i > 1:
            answer = answer + 1
    return answer
```

>=2

포함 불포함
[시작 시각, 끝 시각]

used_tv 리스트

시각	0	1	2	3	4	5	6	7	8		...		24
개수	0	1	2	3	3	2	1	1	1				0

- ① used_tv 리스트에 24 시를 인덱스 번호로 사용하여 총 25 개를 0 으로 초기화 한다.
- ② programs 2 차원 리스트 안에 있는 데이터를 1 차원 리스트로 받아서 시작 시각에서 끝 시각 전까지 개수를 증가시킨다.
- ③ used_tv 리스트의 값이 2 대 이상(TV 를 동시에 두 대 이상 트는 것)인 것의 개수를 구해서 리턴한다. 3 번의 문장을 아래와 같이 리스트의 메소드를 사용해서 작성할 수도 있다.

```
answer=used_tv.count(2) + used_tv.count(3)
```

문제 9 의 정답

주차장에 들어올 수 있는 차량의 수 구하기

```
def solution(day, numbers):  
    count = 0  
    for number in numbers:  
        if number%2 == day%2:  
            count += 1  
    return count
```

```
day = 17  
numbers = [3285, 1724, 4438, 2988, 3131, 2998]  
ret = solution(day, numbers)
```

차량 번호가 적힌 numbers 리스트에서 데이터를 하나씩 읽어 와서 차량 번호와 day 의 짝홀수가 동일하면 count 를 하나씩 증가시킨다.

10

모의고사 3 차 문제 10 번

문제 10 의 정답

자신을 2 로 나눈 값의 개수 구하기

```
def solution(arr):  
    answer = 0  
    for i in arr:  
        if i/2 in arr:  
            answer += 1  
    return answer
```

```
arr = [4, 8, 3, 6, 7]  
ret = solution(arr)
```

리스트에 있는 데이터를 가져와서 현재 데이터를 2 로 나눈 값이 리스트에 들어 있으면 개수를 1 증가시킨다.

※ 주의 : $i/2$ 대신 $\text{int}(i/2)$ 로 작성하면 안됨.
예) $7/2 \rightarrow 3.5$, $\text{int}(7/2) \rightarrow 3$